



F1R3FLY.IO

F1R3GAMES · GAME DESIGN

F1R3Ink

Collective emotional intelligence on a F1R3FLY shard

Version 1 · 8 October 2026

Conforms to F1R3Node-Rust master@3c7872a and the F1R3Games Portal at
F1R3Games main@5937e4e, with F1R3Beat as implemented on f1r3beat@57a3b27

SUMMARY

F1R3Ink is a multiplayer game for collecting a community’s emotional perceptions of its members. It has one move. A player selects another player and *inks* them with a colour that says how they perceive that person at that moment. No colour has a fixed meaning. Each inker owns one stripe on each person they ink, and re-inking changes the stripe’s colour. A person’s stripes, drawn behind their avatar, make up their *freak flag*: how their community sees them. Stripes fade over time unless they are refreshed, so a flag shows what people feel now rather than what they once felt.

Players choose how much they disclose. A player who makes their flag public lets every other player see it in full. A private flag is sealed on the chain, so its colours are readable only by the person flagged and by each inker for their own stripe. An inker chooses, stripe by stripe, whether the ink is attributed to them. Anonymous ink requires a relay, because the shard names the signer of every deploy. Players can also message one another and send one another F1R3Cap, as in F1R3Pix and F1R3Beat.

This document specifies the game against the stack as it stands on 8 October 2026. It replaces the environment installed with the Portal, in which every ink and every inker was public and inks accumulated without limit. On 8 October the author of the game settled who sees which flag, and the substance of seven of the fifteen decisions in Section 18. This document recommends the details of those seven, the other eight decisions, and a way to reward disclosure (§4), for review.

CONTENTS

1	Purpose, sources and conformance	3
1.1	What the game is for	3
1.2	Sources reviewed	3
1.3	Order of authority	4
1.4	Conventions	4
2	What the review found	4
3	The game	5
3.1	Rules	5
3.2	Flags and stripes	6
3.3	Who sees what	6
3.4	Attribution	7
3.5	Re-inking, lifting and history	7
3.6	Colour	8
3.7	Decay	8
3.8	Phases	9
4	Rewarding disclosure	9
5	Architecture	10
5.1	Authority	10
6	The F1R3Ink environment	11
6.1	Configuration	11
6.2	State	11
6.3	Moves	12
6.4	Reads	13
6.5	Order and time	14
6.6	A sketch of ink	14

10 Host protocol additions	17
11 The client	18
11.1 Layout	18
11.2 Behaviour	19
11.3 Styling	19
12 Aggregate views	19
13 Galleries and the play record	20
14 Security and abuse	21
15 Manifest and conformance	22
16 F1R3Gaze	23
17 Development plan	23
18 Decisions	24
18.1 Out of scope for version 1	24

1 PURPOSE, SOURCES AND CONFORMANCE

1.1 What the game is for

F1R3Ink belongs to the F1R3Games suite of collective intelligence games, *E Pluribus Sapiens*. F1R3Pix asks a community to make a picture, F1R3Beat a groove, F1R3SideChat a story and F1R3Skein a melody. F1R3Ink asks it to say how its members feel about one another. What it develops is collective emotional intelligence: a group's capacity to notice and express how its members are perceived, and each member's capacity to see themselves as the group does.

The overview deck of June 2025 introduced the game under the name InkMe. It argues that communities co-evolve with their language and signals, from technical jargon to teen slang, and that this fact has scarcely been applied to how sentiment is expressed and gathered: apart from emojis, mainstream services have explored little of that space. F1R3Ink gathers sentiment through colour, which carries no articulated meaning. The meaning of a colour is whatever a community makes of it. The deck points to the colour studies of Land and of Berlin and Kay for how rich that space is.

The suite's usual premise is that a player's own actions are severely limited, while communication is open to all. F1R3Ink keeps the first half more strictly than any other game in the suite: a player can only colour their own perception of others. Its March design had no messaging and no tokens. The author of the game has since added both (D11), because messages and tokens are social lubricants. A player may find it easier to ink someone they have talked to, and tokens give access to a wide range of resources.

1.2 Sources reviewed

- The F1R3Games overview deck (F1R3GamesOverview.pdf, 20 June 2025), pages 10–12: InkMe's rationale and storyboard. The storyboard shows the two columns, the free colour picker, a stripe labelled with its inker, a badge on the inker's avatar, and a wheel in which each viewer sees only the stripes they themselves gave.
- The original F1R3Ink design session (6 March 2026), which specified the two columns, self-description tags, the wheel, inking, and the freak flag. It produced the prototype in F1R3Ink/UI and F1R3Ink/Shard.
- The design session of 8 October 2026, in which the author of the game settled flag visibility, attribution, messaging and tokens, mutable stripes with history, the palette, decay, and the aggregate views (D1, D2, D3, D4, D5, D11, D13, and the visibility rule of §3.3).
- The F1R3Games repository at main@5937e4e, with F1R3Pix merged, and the f1r3beat branch at 57a3b27. These hold the Portal (Portal/f1r3games-portal), the installed F1R3Ink environment (templates/games/f1r3ink.rho), the game prelude, host protocol 2 with pay, open and declared capabilities, and the Portal's payments and sponsors domains.
- The F1R3Pix design, version 1 (5 October 2026), and the F1R3Beat design, version 2 (6 October 2026), whose structure, conventions and many decisions this document follows.
- F1R3Node-Rust master@3c7872a (8 October 2026), for rho:block:data, which yields the block number, the block's timestamp and its proposer.

- The brand portal, github.com/F1R3FLY-io/f1r3fly-brand-portal (brand kit v1.2), which governs this document's design.

1.3 Order of authority

Where sources disagree, this document follows the order the Portal design set: F1R3Node-Rust master; then Embers; then F1R3Sky; then F1R3Gaze and ign1t10n; then the Portal, F1R3Pix and F1R3Beat as implemented. The deck, the March prototype and its contracts are research. They record what the game is meant to be. This document keeps their rules and their interface, and it does not keep their storage, signing or contract code. Where the deck and the prototype disagree, the deck is followed, and the decisions of 8 October override both.

1.4 Conventions

MUST, MUST NOT, SHOULD and MAY are used as in RFC 2119. Requirements are numbered R1, R2, ... and decisions D7, D1, A *participant* is an address in an instance's participants map in the Portal environment. An *entered* participant is one who has registered with the F1R3Ink environment (enter, §6.3). The *target* of an ink is the person inked, and the *inker* is the person inking. The person a flag belongs to is its *owner*. Times are block timestamps in milliseconds unless stated otherwise.

2 WHAT THE REVIEW FOUND

FINDING 1 Every ink and every inker is public

The installed environment stores each ink in the clear as {from, colour, at}, and its state read returns them all. Any client, and anyone who reads the shard, sees who inked whom and with what. Nothing in the stack supports a private flag or an unattributed ink.

FINDING 2 Inks accumulate; a stripe cannot change

ink appends to the target's list. Each re-ink adds a stripe, so a flag grows without bound and a person who inks often dominates it. In the game as intended, each inker has one stripe on each target, and inking again changes its colour (D3).

FINDING 3 Every inker writes the same key

The target's ink list is a single TreeHashMap entry, (instance, "inks", target), written by everyone who inks that person. Concurrent inks of a popular player conflict on that key. F1R3Pix and F1R3Beat avoid this by giving every key a single writer.

FINDING 4 The prototype's wheel is not the deck's

In the deck's storyboard, Annie's wheel shows only Annie's stripe on Britta, and Britta's own column shows all the stripes she has received. The prototype draws everyone's stripes on every avatar for every viewer. The decisions of 8 October define a third behaviour: public flags are seen in full, and private flags show each viewer only their own stripe (§3.3).

FINDING 5 There are no phases, no pacing and no clock

The installed ink and tags ignore the instance's status, so inking continues after the host closes the round. Nothing limits how often one player inks another. The only time recorded is the deploy timestamp, which the deployer chooses. Decay (D5) needs a time the inker cannot choose.

FINDING 6 There is no history for a gallery

The Portal registers a gallery kind round for F1R3Ink. The environment keeps no block heights and no log, and tags overwrites earlier tags. A round therefore cannot be replayed or verified, and a record cannot show what someone said about themselves when they were inked.

FINDING 7 Colours are barely checked

ink checks only that the colour is seven characters long and starts with #. There is no palette. Tags are unbounded in length.

FINDING 8 The shard names every signer

A deploy is signed, and the environment learns the signer through `rho:deploy:data`. Anyone who reads the shard learns it too. Sealing can hide a colour but not its sender. An ink that is anonymous even to its target therefore cannot be deployed by the inker. It has to be deployed by someone else on the inker's behalf (§8).

FINDING 9 Messaging, payments and sealing exist and can be reused

F1R3Pix added sealed messages (say, mail, outbox), the Portal's `payments.send`, and host protocol 2 with pay and open. F1R3Beat reused them unchanged. F1R3Ink reuses them as well, and the same envelope seals private inks (§7).

3 THE GAME

3.1 Rules

1. An instance is one *round*: a group of participants and a span of time, opened and closed by the host.
2. A participant enters the game by registering a sealing key and choosing whether their flag is public or private. They may describe themselves with a few tags, and may change their tags and their flag's visibility at any time while the round is open.
3. The only move on another person is *ink*. An entered participant sets the colour of their own stripe on another entered participant, choosing from the round's palette. The first ink creates the stripe, and every later ink changes its colour or refreshes it. An inker may also lift their stripe, leaving no colour.
4. When an inker first inks a target, they choose whether the stripe is attributed to them or anonymous.
5. Stripes fade with time unless their inker refreshes them.
6. Any participant may message any set of entered participants, and may send F1R3Cap to any set of participants.

7. Everything that happens is recorded. A round's outcome is its history, and each flag at the close is one frame of it.

There is no win condition. As in F1R3Pix, goals come from outside the game: from what a community wants to learn about itself, or later from tournaments.

3.2 Flags and stripes

A *flag* is the set of stripes on one person. Each stripe belongs to exactly one inker, and each inker has at most one stripe on each target. A stripe has a *location* on the flag, fixed by when its inker first inked the target. It keeps that location for the rest of the round, so a flag reads the same way as it changes. Stripes are drawn left to right across the avatar, behind it, so the avatar always stays in front. The earliest stripe is at the top.

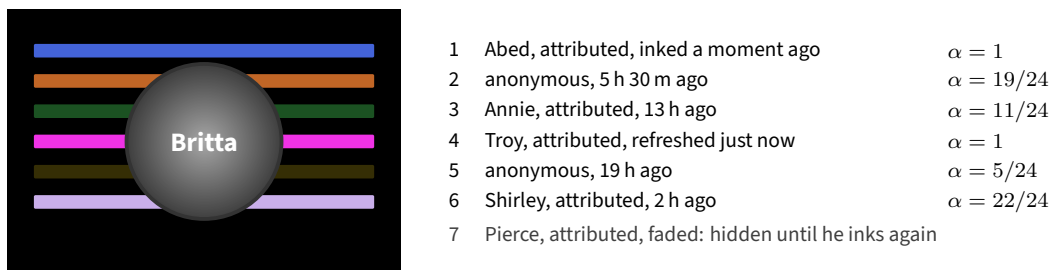


Figure 1: Britta's flag, under the default decay of one step per hour over 24 hours. Stripe locations are fixed by the order of first ink. A faded stripe takes no space, and comes back in its place when refreshed.

3.3 Who sees what

The author of the game settled the rule: *every player can see the full flag of every other player who has declared their flag public*. A private flag is seen in full only by its owner. Every inker can always see their own stripe on anyone.

viewer	a public flag	a private flag
its owner	every stripe, its colour and history; names on attributed stripes	the same
another player	every stripe, its colour and history; names on attributed stripes	the viewer's own stripe only
a reader of the shard, without the game	every stripe and colour; inkers of attributed stripes	when each stripe changed; inkers of attributed stripes; no colours

Anonymous stripes carry no name for anyone. That includes the owner and readers of the shard, subject to the relay's trust (§8).

DECISION D1 What does private mean on the chain?

Settled by the author of the game: each player declares their flag public or private, and public flags are seen in full by everyone. **Recommended for the mechanism:** a private flag's inks are sealed on the chain to two readers, the owner and the inker, with the envelope F1R3Pix uses for messages (§7). A public flag's inks are in the clear. The environment refuses a clear ink on a private flag and a sealed ink on a public one, so the form

always matches the owner's choice at the block the ink lands in.

Changing visibility. Going public discloses the content key of each stripe's *current* sealed colour, and each stripe's earlier history stays sealed. Going private seals inks from then on. It cannot unpublish colours already on the chain in the clear. The game client stops showing them to other players, but a reader of the shard can still find them. The client says so, in one line, when a player switches.

Default. Nothing is preselected. The entry dialogue asks the player to choose, and says in one sentence what each choice shows to whom.

3.4 Attribution

DECISION D2 Who knows who inked whom?

Settled by the author of the game: the inker declares whether their ink is attributed. **Recommended for the details:**

- Attribution belongs to the stripe, and is chosen at its first ink. A stripe cannot switch between attributed and anonymous ink by ink, because an attributed ink would expose every anonymous ink before and after it at the same location.
- An attributed stripe is deployed by the inker. Its name is shown to everyone who can see the stripe.
- An anonymous stripe is deployed by the F1R3Ink relay (§8), so the inker's address never appears on the chain against it.
- An anonymous stripe may later be revealed by its inker. An attributed stripe cannot become anonymous, because its inker is already on the chain.
- The client's default for a new stripe is *attributed*, and the choice is one tap away beside the palette. A player can make anonymous their own default.
- Anonymity among three people is not anonymity. The relay refuses anonymous inks in a round with fewer than anonMin entered participants (default 5), and the host may turn anonymous ink off.

3.5 Re-inking, lifting and history

DECISION D3 What happens when an inker inks the same person again?

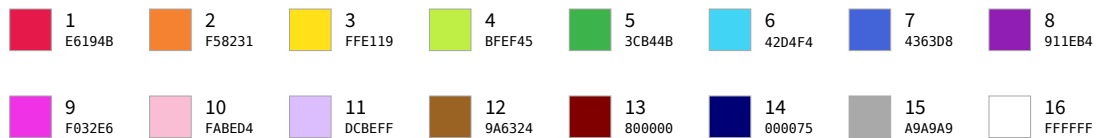
Settled by the author of the game: stripes are mutable, like the cells of F1R3Pix and F1R3Beat. The inker's stripe keeps its location and takes the new colour. Every change is kept, and right-clicking a stripe (or long-pressing it on a touch screen) shows its history of colours. **Recommended additions:** re-inking with the same colour *refreshes* the stripe, which restarts its decay (D5). An inker may *lift* their stripe, which removes its colour until they ink again. The lift stays in the history. Each viewer sees a stripe's history at the same visibility as the stripe itself.

The history popover lists each change newest first: a swatch, the colour's palette number, how long ago, and how faded it was when it was replaced. For an anonymous stripe, the popover shows the history and no name.

3.6 Colour

DECISION D4 Which colours may a stripe take?

Settled by the author of the game: a palette. **Recommended:** the host chooses the palette when creating the round, from 2 to 32 colours. The default is the sixteen below, chosen to be far apart in hue and in lightness. On the chain an ink is a palette *index*, which keeps sealed inks small and makes every colour comparable across a round.



Colours are identified by number and hex value, never by name. The March prototype named its colours (rose, ember, ash, void), and a name such as “ash” suggests a meaning. In F1R3Ink the meaning of a colour belongs to the community that uses it. Numbers also serve players who cannot tell some colours apart: the swatch, its number and its hex value are always shown together.

3.7 Decay

DECISION D5 How do stripes fade?

Settled by the author of the game: colours grow more transparent with each configurable unit of time, until they are refreshed or fade away completely. **Recommended for the details:** the round fixes a unit u and a number of steps k . A stripe last inked at block time t , at time *now*, the opacity

$$\alpha = \max\left(0, 1 - \frac{\lfloor (now - t)/u \rfloor}{k}\right).$$

It loses $1/k$ of its opacity at the end of each unit, and vanishes after k units. Any ink by the stripe’s inker, including the same colour, resets t . A faded stripe keeps its location: it takes no space on the flag, and it returns there when its inker inks again. The host may choose no decay.

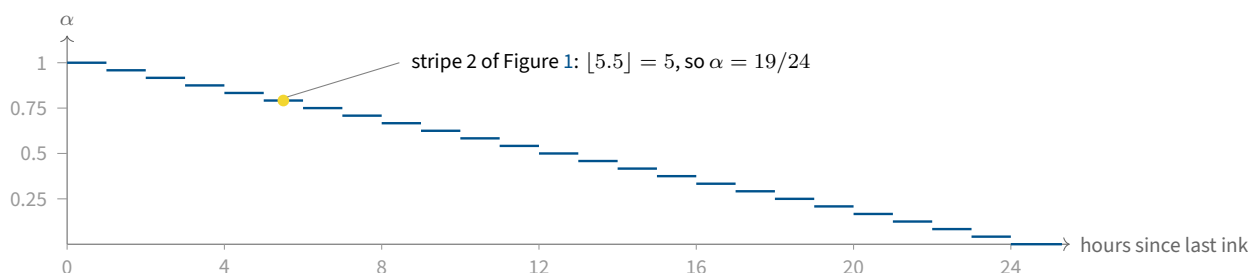


Figure 2: Opacity under the default decay, a *day*: one step per hour, over 24 steps.

preset	unit u	steps k	gone after	suits
an evening	10 minutes	12	2 hours	a game played in one sitting
a day (default)	1 hour	24	24 hours	a team or a class over a day
a week	6 hours	28	7 days	a community over a week
a season	1 day	90	90 days	a long-lived community
never	—	—	—	a round where every ink stands

Decay is not computed on the chain. The chain records when each stripe was last inked, by the block's timestamp, and every reader derives α from it. All readers at the same block therefore agree. Between blocks, the client advances *now* by its own clock, but never past the next block it sees. When a round closes, *now* stops at the closing block. A closed round's flags stay as they were at the close.

3.8 Phases

F1R3Ink uses the Portal's instance status (lobby, active, closed), set only by the host.

DECISION D6 What may players do in each phase?

Recommendation.

	enter	tags, visibility	ink	message	pay	read
lobby	yes	yes	no	yes	yes	yes
active	yes	yes	yes	yes	yes	yes
closed	no	no	no	no	no	yes

The lobby lets players gather, describe themselves, choose their visibility and talk before anyone is inked, so that a round starts together. Closing freezes every flag, its decay included.

DECISION D7 How large is a round?

Recommendation: the host sets a capacity when creating the round, from 3 to 64 participants, with a default of 24. A flag has at most one stripe per other participant, so capacity bounds both a flag's height and the cost of reading every flag (§6.4). The upper bound is provisional, and Phase 4 measures it (§17). A participant who leaves through the Portal keeps the stripes they gave, which go on fading. Their own flag stays as it was, and no one can ink them until they rejoin.

4 REWARDING DISCLOSURE

The game works only if enough flags are public. A private flag still tells its owner how the community sees them, but it teaches the community nothing about itself. The author of the game asked what, beyond a more fulfilling game, might encourage disclosure. The proposals below avoid the incentive that would damage the game, which is paying for colours. A reward for disclosure should reward *showing* a flag, never what is on it, so that no one has a reason to ink for effect.

DECISION D8 What rewards a public flag?

Recommendation: the first three in version 1, the fourth as a host option, the fifth later.

- The commons.** The community spectrum (§12) is built from public flags only, and it says so in its header: "14 of 22 flags". Going public visibly adds a person to the picture the community has of itself. Going private visibly takes them out.
- The portrait.** Only a flag's owner may publish it to the gallery as a flag play (§13). The portrait is a record of how a community saw someone, which others can view and react to. A private flag can be published too, which discloses it, and the act of publishing is the owner's to choose.
- The conversation.** The owner of a public flag can see who holds each attributed stripe, and so can everyone else. That makes the flag an opening for conversation, by message, about why someone chose a colour.

Many players will find that the richest part of the game.

4. **Show to see** (host option, reciprocity, off by default). Under this option, only a player whose own flag is public sees other public flags in full. A player whose flag is private sees only their own stripes and the counts. This is the strongest incentive here, but it narrows the settled rule that every player sees every public flag. It is therefore offered to hosts and not imposed. It is also enforced only by the client: public colours are on the chain in the clear, and a reader of the shard can see them regardless.
5. **A stipend for disclosure** (later). A sponsor could fund a stipend paid at the close to every player whose flag was public for the whole round, through the Portal's sponsors domain. The stipend pays for showing a flag, not for its colours. It needs the Portal's sponsorship terms to ask the game environment about disclosure, which they cannot do today, so it is left for a later version.

5 ARCHITECTURE

F1R3Ink adds one new kind of component to what F1R3Pix and F1R3Beat use: a *relay*, which deploys anonymous inks for their inkers. Everything else is one game environment on the shard, one game client framed by the Portal shell, and the Portal's existing payments domain and host protocol, which gains a relay method.

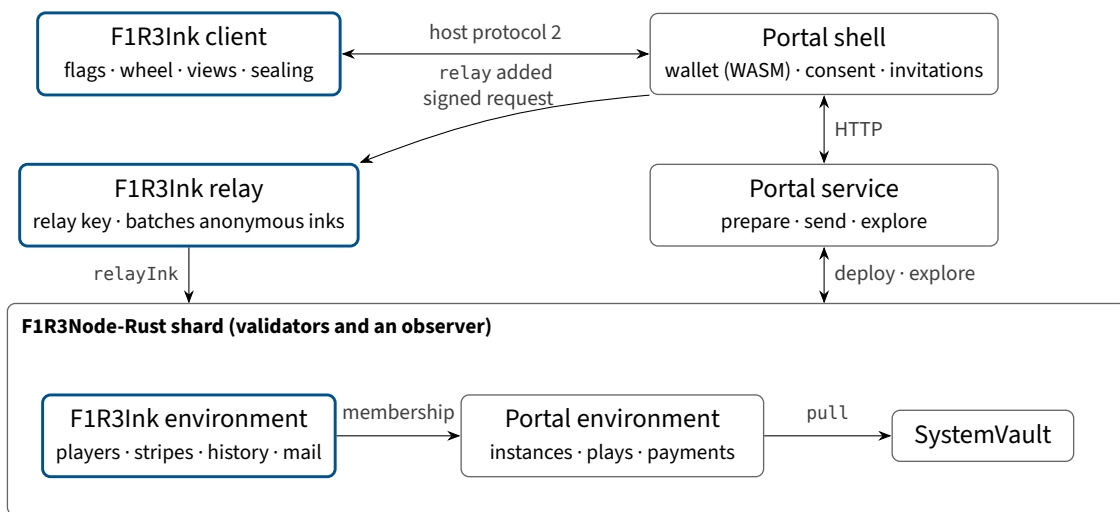


Figure 3: Components. Outlined in dark sky: what this design adds or replaces.

5.1 Authority

- The game client never sees a key. It reaches identity, the shard, payment and the relay only through the host protocol.
- The F1R3Ink environment reads two things and nothing else. The first is the caller's address, which it obtains from `rho:deploy:data` through `rho:vault:address`, and the second is the block's number and timestamp, from `rho:block:data`. It never touches a vault.
- Funds move only through the Portal environment's `payments . send`, signed only from the Portal shell after a prompt.

- Sealed colours are sealed in the game client and opened by the wallet. The chain holds only ciphertext.
- The relay holds one key, the *relay* key, which the F1R3FLY.io Cooperative names in the F1R3Ink environment, as it names F1R3Beat's breeder. That key can write and reveal anonymous stripes, and nothing else. Unlike the breeder, the relay is trusted: it knows who holds each anonymous stripe (§8).

6 THE F1R3INK ENVIRONMENT

The environment replaces the installed `f1r3ink.rho` body inside the shared game `prelude.rho`. It keeps the prelude's machinery: installation under the game's registry key with `rho:registry:insertSigned:secp256k1`, version-checked upgrade, `who`, `member`, `instanceOf` and the `TreeHashMap` store. Installing it raises the environment's version. The URI is unchanged, every call template's hash changes, and the F1R3FLY.io Cooperative re-registers the manifest (§15).

6.1 Configuration

The configuration is the `config` map the host passes to the Portal's `instances.create`. It is read from the Portal's instance record and fixed at creation.

```
{ "capacity":      24,                // 3 ..= 64                                (D7)
  "palette":      ["#E6194B", ...],    // 2 ..= 32 distinct "#RRGGBB", upper case (D4)
  "decay":        { "unit": 3600000, "steps": 24 }, // ms and count, or Nil                (D5)
  "minInterval":  60000,                // ms between inks on one stripe        (D9)
  "anonymous":    true,                 // may stripes be anonymous?            (D2)
  "anonMin":      5,                   // entered participants needed for anonymity (D2)
  "reciprocity":  false,                // "show to see"                        (D8)
  "messageLimit": 2048 }                // bytes of sealed envelope per message (sec. 9)
```

R1 The environment **MUST** refuse every move on an instance whose configuration is missing or out of range, answering (`false`, `"bad config"`). It **MUST NOT** substitute defaults. Defaults are applied by the client that creates the instance, where the host can see them.

6.2 State

Every key is scoped by instance, and every key has exactly one writer.

key	value	writer
("p", i, a)	{"pk": "public" "private", "tags": [...], "veil": [sid, ...], "at": [h, t]}	<i>a</i>
("plog", i, a)	list of [h, t, "flag", v] and [h, t, "tags", tags], oldest first	<i>a</i>
("disc", i, a)	list of [sid, seq, K]: content keys <i>a</i> disclosed on going public	<i>a</i>
("s", i, b, sid)	stripe: {"by": a Nil, "first": [h, t], "last": [h, t], "seq": n, "ink"}	inker, or relay
("sh", i, b, sid)	the stripe's history: list of [h, t, ink]	inker, or relay
("o", i, a)	the set of targets <i>a</i> has inked	<i>a</i>
("o", i, "relay")	map from target to the set of anonymous handles on them	relay
("out", i, a)	sealed messages sent by <i>a</i> , as F1R3Pix	<i>a</i>

Here *b* is the target, *h* a block number and *t* a block timestamp. A stripe id *sid* is the inker's address for an attributed stripe, or ["anon", handle] for an anonymous one. An ink is {"c": *n*} (palette index *n*, in the clear), {"sealed": bytes} (an envelope, §7), or Nil (lifted).

R2 Every key MUST have the single writer shown. An inker writes only their own stripes and their own index, and the relay writes only anonymous stripes and its own index. Two players inking the same target therefore never contend for a key, which fixes Finding 3. The environment keeps no instance-wide counter or log. Global order is recovered when reading (§6.5).

6.3 Moves

Moves are play templates. Within the allowance granted at launch they are signed without a prompt. Each answers (true, value) or (false, reason).

enter(instance, pk, flag). Registers the caller. *pk* is their uncompressed secp256k1 public key, used to seal inks and messages to them, and *flag* is "public" or "private".

- Refuses unless the caller is a participant and the status is not closed.
- Refuses unless `rho:vault:address's "fromPublicKey"` applied to *pk* yields the caller's address.
- If the caller has already entered, answers their record unchanged.
- Refuses with "full" if the round already has capacity entered participants.

tags(instance, tags). Replaces the caller's tags and appends the change to their log. Refuses unless the caller has entered and the status is not closed. A list may hold at most 12 tags, each 1 to 24 characters with no control characters.

visibility(instance, flag, keys). Sets the caller's flag to "public" or "private" and appends the change to their log. Going public requires keys, the content key of the current envelope of every sealed stripe on the caller's flag. The environment appends them to ("disc", i, a). It cannot check them, because it cannot run AES-GCM. Every client checks a disclosed key against its envelope, and a stripe whose key is missing or wrong is drawn veiled, with a mark saying the owner did not disclose it. Going private takes Nil.

ink(instance, target, ink). Sets the caller's attributed stripe on target.

- Refuses unless the caller and the target are both entered participants, the target is not the caller, and the status is active.

- Refuses unless `ink` has the form the target's flag requires: `{"c": n}` with $0 \leq n < |\text{palette}|$ for a public flag, or `{"sealed": bytes}` of at most 320 bytes for a private flag. `Nil` (a lift) is accepted for either.
- Reads the block's number h and timestamp t . If the stripe exists, refuses with "too soon" unless $t \geq t_{\text{last}} + \text{minInterval}$.
- Writes the stripe (creating it with `first = [h, t]` if it is new), sets `last = [h, t]`, increments `seq`, appends `[h, t, ink]` to its history, and adds the target to the caller's index. Answers `{"sid", "seq", "h", "t"}`.

veil(instance, sids). Sets the list of stripes on the caller's own public flag that the client hides from other players (D15). The stripes' colours stay on the chain.

say(instance, to, envelope). As F1R3Pix's `say`, with recipients drawn from the entered participants.

Relay moves. Two moves are signed only by the relay key, from the relay, and are not game templates.

- `relayInk(instance, batch)`, where `batch` is a list of up to 64 `[target, handle, ink]`. Each entry is checked as `ink` checks it, against the stripe `["anon", handle]`, and also against anonymous and `anonMin`. Entries are independent: the move answers one result per entry.
- `relayReveal(instance, target, handle, address)` sets the `by` of an anonymous stripe, revealing its inker. The stripe keeps its `id`, location and history.

`setRelay(address)`, signed only by the holder of the F1R3Ink registry key, names the relay, as F1R3Beat's `setBreeder` names the breeder.

6.4 Reads

Reads are explore deploys served by the observer. They cost the reader nothing and need no signature.

template	answers
<code>f1r3ink.players(instance)</code>	every entered participant: <code>pk</code> , <code>flag</code> , <code>tags</code> , <code>veil</code> , at
<code>f1r3ink.flags(instance)</code>	every stripe, grouped by target: <code>sid</code> , <code>by</code> , <code>first</code> , <code>last</code> , <code>seq</code> and the current <code>ink</code> ; with every ("disc", ...) list
<code>f1r3ink.history(instance, target, sid, from)</code>	a stripe's history from position <code>from</code> , up to 256 entries
<code>f1r3ink.log(instance, from, to)</code>	every <code>ink</code> , tag change and visibility change with $\text{from} \leq h < \text{to}$, in the order of §6.5
<code>f1r3ink.mail(instance, address, cursor), outbox(...)</code>	as F1R3Pix

`flags` reads each participant's index and then only the stripes that exist, so it touches $n + S$ keys for n participants and S stripes. At the default capacity that is at most $24 + 24 \cdot 23 = 576$, and at 64 it is at most 4,096. Phase 4 measures whether the upper bound holds as a single read.

R3 Every read result **MUST** carry the number, hash and timestamp of the block it reflects. The Portal service adds the timestamp, which it reads from the block's header. The client takes `now` for decay from it (D5).

6.5 Order and time

An ink is stamped with the number and timestamp of the block that includes it, from `rho:block:data`. Decay runs on the block's timestamp and not on the deploy's, because the deploy timestamp is chosen by the inker, who could otherwise keep a stripe artificially fresh. The block timestamp is chosen by the proposing validator. History is ordered by block number, then by deploy timestamp within the block, then by target, then by stripe id.

6.6 A sketch of ink

```
contract env(@"ink", @instance, @target, @ink, ret) = {
  new whoCh, rCh, cCh, gCh, sCh, ack1, ack2, ack3 in {
    who!(*whoCh) |
    for (@by, _ <- whoCh) {
      // config valid? both entered participants? not self? "active"?
      // form of ink matches the target's flag, and index in the palette?
      inkReady!(instance, by, target, ink, *rCh) |
      for (@ok, @why <- rCh) {
        if (not ok) { ret!((false, why)) }
        else {
          clock!(*cCh) | // rho:block:data
          gapOf!(instance, *gCh) | // minInterval
          TreeHashMap!("get", state, key, *sCh) | // key = ("s", instance, target, by)
          for (@h, @t <- cCh & @gap <- gCh & @stripe <- sCh) {
            match stripe {
              Nil => {
                put!(key, {"by": by, "first": [h, t], "last": [h, t],
                  "seq": 1, "ink": ink}, *ack1)
              }
              _ => {
                if (t < stripe.get("last").nth(1) + gap) {
                  ret!((false, "too soon"))
                } else {
                  put!(key, stripe.set("last", [h, t])
                    .set("seq", stripe.get("seq") + 1)
                    .set("ink", ink), *ack1)
                }
              }
            }
          } |
          for (_ <- ack1) {
            append!(("sh", instance, target, by), [h, t, ink], *ack2) |
            addTarget!(("o", instance, by), target, *ack3) |
            for (_ <- ack2 & _ <- ack3) {
              ret!((true, {"sid": by, "h": h, "t": t}))
            }
          }
        }
      }
    }
  }
}
```

```
}

```

inkReady, clock, put, append and addTarget are prelude-level helpers. gapOf reads minInterval from the configuration, and key stands for ("s", instance, target, by). relayInk runs the same body for each entry, with ["anon", handle] as the stripe id and "by": Nil.

6.7 Pacing

DECISION D9 How often may one person ink another?

Recommendation: at most once per minInterval per stripe (default one minute of block time), enforced on the chain. The client also coalesces inks, as F1R3Pix coalesces paints: while one ink is in flight, further choices replace a single queued ink. Browsing the palette therefore costs one deploy. The interval stops one player from flickering a stripe to draw attention, and it bounds the history at roughly one entry per minute per stripe.

7 SEALING PRIVATE FLAGS

A private flag's inks use the envelope F1R3Pix defined for messages, with two changes.

1. **Domain.** The info and aad strings are "f1r3ink/ink/v1", and the aad binds the instance, the target, the stripe id and the stripe's next seq. A sealed ink cannot be replayed onto another stripe or into another position of its own history.
2. **Unlabelled wraps.** A message envelope labels each wrap of the content key with its recipient's address. A sealed ink has exactly two wraps, one for the target and one for the inker, and both are unlabelled and in random order. Each reader tries both. An anonymous stripe would otherwise name its inker in its own envelope. All sealed inks use unlabelled wraps, so that attributed and anonymous inks look alike.

The plaintext is the palette index as one byte, padded to a fixed 16 bytes, so every sealed ink has the same length. The envelope comes to about 250 bytes, within the 320-byte bound of ink. Phase 1 fixes the exact layout against test vectors.

R4 The host's open MUST accept envelopes with unlabelled wraps, trying each wrap with the active key, and MUST bind the hosted game and instance as F1R3Pix's R4 requires. It answers not-addressed when no wrap opens.

Disclosure. Going public discloses the content key K of each stripe's current sealed ink, which the owner obtains by opening it. Anyone holding K can decrypt that one envelope and check it against the chain, and nothing else. The stripe's earlier inks stay sealed. A disclosed colour is as verifiable as a clear one.

8 THE RELAY

Finding 8 leaves one way to make an ink anonymous on a public chain: someone other than the inker deploys it. The relay is that someone. It is a small service, run by the F1R3FLY.io Cooperative beside the Portal service, that holds the relay key and pays the phlo for anonymous inks.

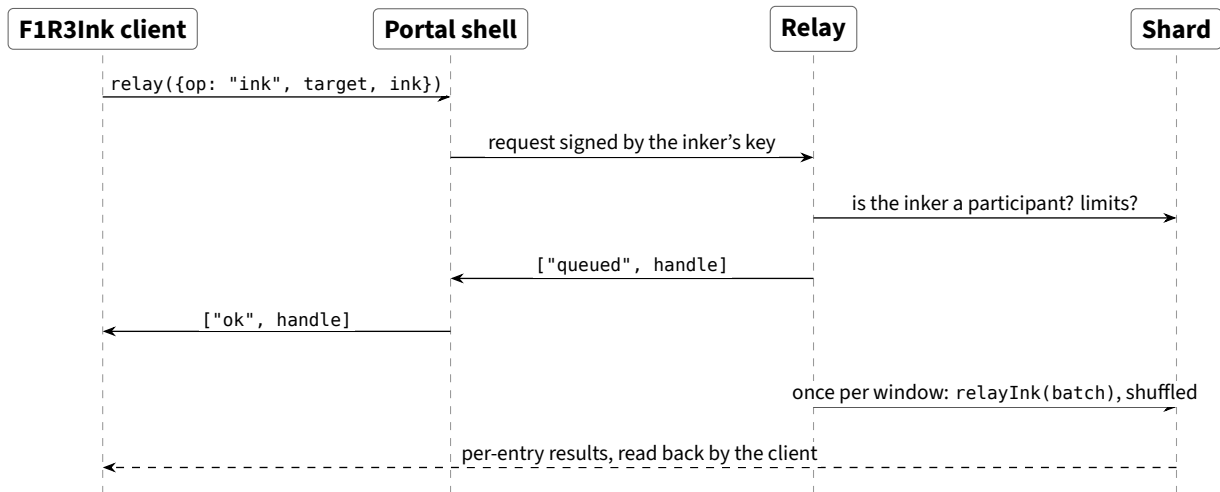


Figure 4: An anonymous ink. The inker's address goes to the relay and never to the chain.

Requests. The game asks the shell to relay a request. The shell signs it with the inker's identity key under the domain "f1r3games/relay/v1" together with the game id, the instance and the relay's URL from the manifest, and posts it to the relay. The relay checks the signature and the inker's participation, and queues the ink.

Handles. The relay derives each stripe's handle as

$$handle = \text{HMAC-SHA256}(s_{\text{relay}}, "f1r3ink/handle/v1" \parallel instance \parallel target \parallel inker)_{0..16},$$

with a secret s_{relay} that it keeps. The same inker on the same target always gets the same stripe, and the relay keeps no table. The inker's client learns its handle from the relay's answer, and can ask for it again with a signed request.

Batching. The relay submits at most one `relayInk` per window, three blocks by default, carrying every ink queued in that window in random order. That blurs when an anonymous ink was given to within a window. It cannot hide it from a target who knows that only one person was online.

Limits. The relay applies `minInterval` before queueing, so a refused ink is refused at once. It caps each inker at 30 relayed inks per hour, so that the Cooperative's phlo cannot be drained through it. If an entry is refused because its target changed visibility after the request, the relay returns it to the inker's client, which seals or unseals the ink and asks again.

DECISION D10 How much must players trust the relay?

Recommendation: accept a trusted relay in version 1, and say so. The relay sees who inks whom, and for public flags, with which colour. It cannot read a sealed colour, and it cannot forge an ink, because every request carries the inker's signature, which the relay keeps and which an auditor can demand. The trust it needs is the

trust the Cooperative already holds as the registry's governor. Removing that trust needs anonymous credentials, with which a participant proves membership of the round without saying which member they are. That means ring signatures, blind signatures or a zero-knowledge membership proof, and F1R3Node-Rust offers none of them as a system process today. The client explains this in the anonymity toggle's help: an anonymous ink is hidden from every player and every reader of the shard, but not from the F1R3FLY.io Cooperative.

R5 relay **MUST** be a declared capability, and the shell **MUST** post only to the relay URL in the hosted game's registered manifest. A game frame cannot use it to obtain the inker's signature for any other purpose.

9 MESSAGES AND PAYMENTS

DECISION D11 How do players talk and pay?

Settled by the author of the game: include both. Recommended: exactly as F1R3Pix (its D6, D7 and D10) and F1R3Beat (its D8). Messages are on the chain and sealed to their recipients and the sender, with the domain string "f1r3ink/msg/v1". Payments go through the Portal environment's `payments.send`, signed only from the shell after a prompt, with *each* as the default and *split* on request. The manifest declares "capabilities": ["pay", "open", "relay"].

The same requirements apply: the host binds the game id and instance into `open`, refuses a pay that names a non-participant before prompting, and never signs `payments.send` within an allowance. Messages to an anonymous inker are out of scope for version 1 (§18.1). A player can message only the people they can name.

10 HOST PROTOCOL ADDITIONS

method	behaviour
<code>relay({op, ...})</code>	New, declared. Signs the request with the active key under the domain of R5 , posts it to the manifest's relay URL, and answers the relay's reply. Counts against the launch allowance like a play move, and never prompts within it. Errors: <code>no-relay</code> , <code>relay</code> , <code>allowance</code> .
<code>open({envelope})</code>	Extended to unlabelled wraps (R4). Unchanged otherwise.

The relay's operations are `ink` (target, ink), `reveal` (target) and `handles` (instance), which lists the caller's own handles. The Portal environment gains one record: when `instances.setStatus` changes an instance's status, it records the block number and timestamp under `statusAt`, so that every reader agrees on when the round closed (D5). Neither addition is specific to F1R3Ink.

11 THE CLIENT

The client is a web page served from the game's own origin and framed by the Portal. Like the F1R3Pix and F1R3Beat clients, it has a framework-free TypeScript core holding every operation and a React layer that only renders, so that each core operation can later become a f1r3lang process in F1R3Gaze.

11.1 Layout

The page keeps the deck's two columns. Messaging and tokens are added to the left column, beneath the player's own flag, and do not get a third column.

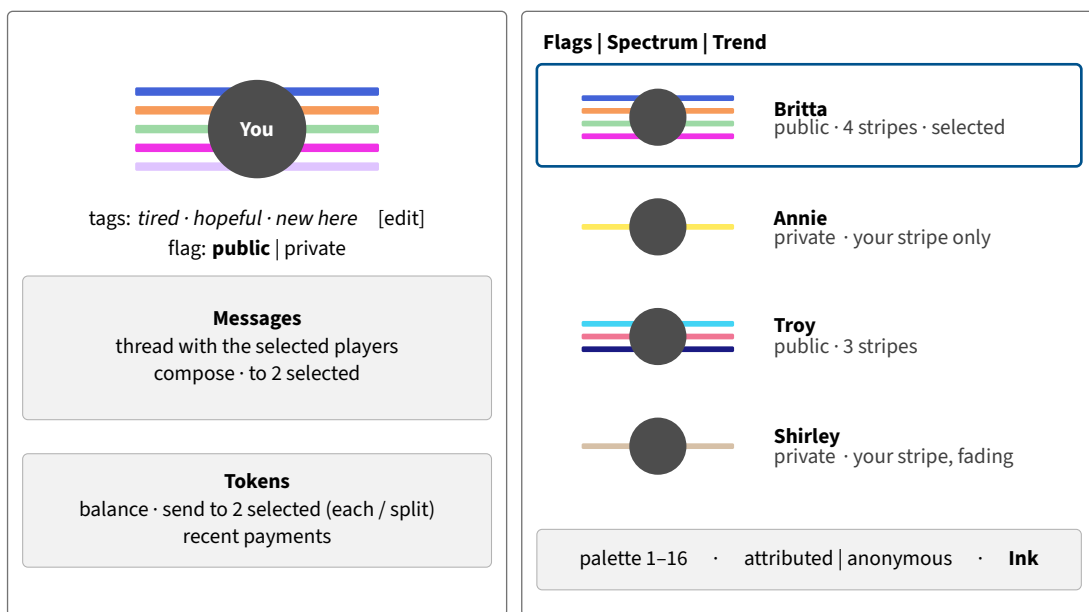


Figure 5: The two columns. Left: you, your flag, your tags and your visibility, then messages and tokens. Right: the wheel, which can switch to the aggregate views, with the ink control for the selected player.

Left column: you. Your avatar with your full flag behind it, your tags (edited in place), and your flag's visibility, with a one-line reminder of who can see it. Beneath them are the message thread with the selected players and the token controls, which work exactly as in F1R3Pix: the send control targets the selected players and shows *each* or *split* with the total.

Right column: everyone else. A vertical wheel of the other entered players' avatars, drawn from Portal profiles. The avatar at the wheel's focus is full size, and avatars shrink with distance from it. Each avatar carries the stripes the viewer may see (§3.3): a public flag in full, a private flag as the viewer's own stripe alone, with a small lock. The viewer's own stripe on a public flag is marked with a notch at its left end. Selecting players drives the message and payment recipients. Selecting exactly one player opens the ink control beneath the wheel: the palette, the attribution toggle for a new stripe, and **Ink**. The wheel's header switches the column between *Flags* (the wheel), *Spectrum* and *Trend* (§12), and holds *Invite*, which opens the Portal's shared contacts dialogue.

DECISION D12 In what order does the wheel list players?

Recommendation: by the most recent interaction with the viewer (an ink given, an attributed ink received, a message or a payment), with *by name* and *newest stripe on you* one tap away. F1R3Ink has no board, so there is no distance to sort by. Recent interaction keeps the people a player is engaged with at the focus.

11.2 Behaviour

- **Notification.** When a stripe on you changes, its inker's avatar in the wheel carries a badge if the stripe is attributed, as in the deck. If it is anonymous, the badge goes on your own avatar.
- **Names.** An attributed stripe shows its inker's name on hover, in the history popover, and in a *names* overlay that labels every stripe at once.
- **History.** Right-click or long-press a stripe to open its history (D3). Opening a sealed history entry calls open once per entry, and the result is cached for the session.
- **Freshness.** The client reads `flags`, `players`, `mail` and `payments` . `list` once per new block and recomputes opacity between blocks (R3). Every read carries its block, and the client never shows a state older than the one on screen.
- **Pending inks.** An ink shows at once with a pending mark and settles when a block includes it. Anonymous inks settle when the relay's batch lands.
- **Allowance.** At launch the Portal grants an allowance for `enter`, `tags`, `visibility`, `ink`, `veil`, `say` and `reLay`, with a budget and an expiry. Payments are never inside it.
- **Accessibility.** The information here *is* colour, so every colour is also given by number and hex value, on hover and to screen readers. Opacity is also given as a number of steps remaining. Flags can be read stripe by stripe with the keyboard, in location order.
- **Small screens.** Below 900 pixels wide, the two columns become two tabs, with the wheel as the default. The selection persists across tabs.

11.3 Styling

The chrome follows the F1R3FLY brand system: a black surface, League Spartan and Source Sans 3, and Yellow and Sky as accents. The flags carry the players' colours and nothing else. Brand colours never appear in a flag, so that no stripe can be mistaken for the interface. Game-level marks are still open across the suite, so the client uses the F1R3FLY.io system, as F1R3Pix and F1R3Beat do.

12 AGGREGATE VIEWS

The deck says each member can see the volume of the signal and how it trends. The author of the game settled that players may choose either of two views.

DECISION D13 What aggregates do players see?

Settled by the author of the game: both, chosen by the player. Recommended for the details: each view is built only from what its viewer may see (§3.3), so an aggregate never leaks a private colour.

Spectrum: the community now. A single band divided among the palette's colours, each in proportion to the summed opacity of that colour across every visible stripe at *now*. The visible stripes are the stripes of public flags, the viewer's own flag, and the viewer's own stripes. Beneath the band is a sparkline of the volume of the signal: inks per unit of decay, across the whole round. Volume counts every ink, sealed and anonymous ones included. The chain shows when each ink happened, though not its colour, so counting does not reveal what is hidden. The header gives how many flags the band is built from (D8).

Trend: one person over time. For the selected player, or for the viewer, a stream graph over the round. The horizontal axis is time, and at each time the stream's colours are that flag's opacity-weighted colour shares. Beneath it is the volume of inks that person received. It is available for any flag the viewer may see in full. For a private flag, other players see only its volume.

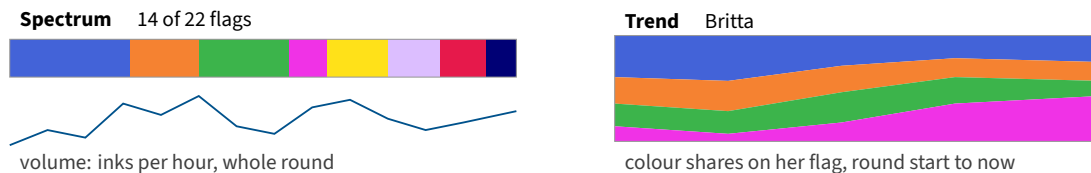


Figure 6: The two aggregate views, as sketches.

13 GALLERIES AND THE PLAY RECORD

DECISION D14 What does F1R3Ink publish?

Recommendation: two kinds, linked.

- round, as the Portal already registers it: the history of a span of the round, by default the whole of it. Any participant may publish one, and the host is offered it at the close. It holds only what is already public on the chain, so publishing it discloses nothing. Private flags appear as their volume and their sealed events.
- flag, a portrait: one person's flag over a span, with its history. Only the flag's owner may publish it. A portrait of a private flag includes the content keys of the inks it shows, so it discloses them, and anyone can verify them against the chain. Portraits link to their round with `linkPlays`, in both directions.

Encoding. The round body is a byte array that rebuilds from `f1r3ink.log`:

```
"F1NK" u8 version = 1 u32 from u32 to u8 unit-steps u32 unit-ms
u8 P P x 3 bytes the palette
varint N N x 20 bytes addresses, in order of first appearance
varint A A x 16 bytes anonymous handles, in order of first appearance
varint E E x (varint dh, varint dt_ms, u8 type, payload) events in history order
  type 0 visibility u8 player, u8 public?
  type 1 tags u8 player, varint len, UTF-8 tags joined by U+001F
  type 2 ink u8 target, stripe, ink
  type 3 veil u8 player, varint n, n x stripe
```

```

    type 4 reveal      varint handle, u8 player
    stripe := u8 0, u8 player | u8 1, varint handle
    ink    := u8 0, u8 colour | u8 1 (lifted)
           | u8 2, u8 colour or 255, 32-byte envelope hash

```

A sealed ink carries its colour only if its owner disclosed it within the span. The flag body uses the same layout restricted to one target, with the content keys appended as varint K , $K \times$ (event index, 32 bytes). The header's preview is the frame at the span's end: for each public flag, its stripes as colour and remaining steps. That is at most two bytes per stripe.

A play is *verifiable*. Anyone can rebuild its body from log at its recorded block, check every disclosed key against its envelope, and compare hashes.

Engagement. Views and likes go through the Portal's engage and order the gallery, as in F1R3Pix. F1R3Ink does not breed plays.

14 SECURITY AND ABUSE

Everything in F1R3Pix's table applies to messages and payments. The rows below are the threats F1R3Ink adds.

threat	answer
Writing someone else's stripe	Impossible: ink writes only the caller's own stripe. Only the relay writes anonymous stripes.
Flooding a flag	One stripe per inker per target, and <code>minInterval</code> between inks (D9).
Hurtful inks	A colour carries no fixed meaning, and the game has no words in a flag. A target may <i>veil</i> stripes on their public flag from other players' view (D15). Their colours stay on the chain, and the target always sees them. A private flag shows a target's stripes to no one else.
Unmasking anonymous inkers	The chain never carries the inker's address against an anonymous stripe. Timing is blurred by batching, and anonymity is refused below <code>anonMin</code> . The relay knows the inker (D10). A small round leaks through who was online, and the client says so.
A dishonest relay	It cannot forge an ink, because it holds the inker's signed request. It could drop or delay inks. Clients read their anonymous inks back and report any that never land. The Cooperative can rotate the relay key.
Fresh-looking stripes	Decay runs on block time, which the inker does not choose (§6.5).
Replaying a sealed ink	The envelope's <code>aad</code> binds the instance, target, stripe and sequence number.
False disclosure	Each disclosed key is checked against its envelope, and a stripe whose key fails is drawn veiled and marked.
Payments for inks	Payments are public metadata, as in F1R3Pix, so payments followed by inks are visible to all. A colour has no fixed meaning to buy. The game does not forbid deals, just as F1R3Pix does not.
Metadata	Who inks a private flag, and when, is public for attributed stripes. Volume is public for every flag.

DECISION D15 May a target hide stripes on their own flag?

Recommendation: yes, from other players' view of a public flag only, by veil. A public flag is a disclosure, and its owner should not have to disclose a stripe they find hurtful in order to disclose the rest. A veiled stripe is drawn as a neutral grey bar with no colour, so its presence stays honest. Veiling does not remove anything from the chain, and it does not change what the owner sees. The client tells the owner both of these things.

15 MANIFEST AND CONFORMANCE

```
{ "id": "f1r3ink", "name": "F1R3Ink",
  "tagline": "Say how you see each other, in colour.",
  "entry": "<entry-base>/f1r3ink/", "platforms": ["web"],
  "env": "rho:id:...", // unchanged URI; environment version raised
  "templates": [ "f1r3ink.enter", "f1r3ink.tags", "f1r3ink.visibility",
                 "f1r3ink.ink", "f1r3ink.veil", "f1r3ink.say", // play
                 "f1r3ink.players", "f1r3ink.flags", "f1r3ink.history",
                 "f1r3ink.log", "f1r3ink.mail", "f1r3ink.outbox" ], // explore
  "capabilities": ["pay", "open", "relay"],
  "relay": "<relay-base>/f1r3ink/",
  "galleries": [
    { "kind": "round", "renderer": "<entry-base>/f1r3ink/preview/round.html" },
    { "kind": "flag", "renderer": "<entry-base>/f1r3ink/preview/flag.html" } ],
  "contactsDialogue": true, "readerTier": false, "minProtocol": 2 }
```

The installed template `f1r3ink.state` is retired. `relayInk`, `relayReveal` and `setRelay` are not game templates: they are signed only by the relay key or the registry key, and are absent from the manifest.

Conformance tests extend the Portal's, F1R3Pix's and F1R3Beat's:

- the environment and every template parse with the node's own parser and normalise with its compiler, as the F1R3Beat work found necessary. Every play template binds no deployer authority;
- decay vectors: for a set of stripes and times, the Rust and TypeScript opacity functions agree, including at every step boundary and at the close;
- sealed inks seal in TypeScript and open in the Rust wallet with unlabelled wraps, refuse to open under another game, instance, stripe or sequence number, and verify under a disclosed key;
- relay handles: given a secret and inputs, the relay's handle matches the vector, and a signed request with a wrong domain is refused;
- the round and flag encodings round-trip against shared vectors in Rust and TypeScript;
- end to end against the mock node: enter, tag, ink attributed and anonymous, refresh, lift, switch visibility with disclosure, veil, say, open, pay, close, and publish a round and a portrait that both rebuild from the chain.

16 F1R3GAZE

F1R3Ink needs the pay and open rows that F1R3Pix adds to the Portal's GAZE-MAPPING.md, with open extended to unlabelled wraps. It adds one capability: relay, a signature under a fixed domain, bound to one URL named in the game's manifest, which never yields the key and cannot be pointed elsewhere. Like open, it is a wallet operation with a caller-fixed purpose. That is the shape F1R3Gaze's wallet capabilities already take. GAZE-MAPPING.md should gain the row when Phase 2 lands.

17 DEVELOPMENT PLAN

phase	work	done when
1 · Environment	New f1r3ink.rho body: enter, tags, visibility, ink, veil, say, the relay moves and setRelay; reads; configuration; decay and encoding vectors.	Parses and normalises; template rules pass; mock-node tests of every move and read.
2 · Sealing and relay	Ink envelope with unlabelled wraps in TypeScript and the Rust wallet; open extended; host relay; the relay as a service beside f1r3games-service; statusAt in the Portal.	Seal, open and disclose vectors pass; an anonymous ink lands from a signed request.
3 · Client	Core and views: flags, wheel, ink control, history popover, tags, visibility; messages and tokens from the F1R3Pix core; spectrum and trend; accessibility; tabs.	A full round against the mock node with six simulated players, public and private, attributed and anonymous.
4 · Live shard	Install on an ign1t10n shard; measure phlo for each move, the flags read at 24 and 64 participants, and the relay's window.	One live round of at least eight players, start to close, with decay visibly at work.
5 · Galleries	round and flag encodings, publishing, renderers, engagement.	A closed round publishes both kinds, and both rebuild and verify from the chain.
Later	Trustless anonymity; replies to anonymous stripes; disclosure stipends; partial disclosure; tournaments; the F1R3Gaze version.	

Phases 1 and 2 can proceed together, and Phase 3 can begin against stubs. The client reuses F1R3Pix's core for messages, payments and the wheel, and F1R3Beat's pattern for reading the gallery from inside the game.

18 DECISIONS

	question	decision or recommendation
D1	Settled. Flag visibility	Public or private per player; public flags seen in full by everyone. <i>Recommended:</i> private inks sealed to owner and inker; going public discloses current colours; no preselected default.
D2	Settled. Attribution	Declared by the inker. <i>Recommended:</i> per stripe, at first ink; attributed by default; anonymous via the relay; reveal one way; anonMin 5.
D3	Settled. Re-inking	One mutable stripe per inker per target, with its history on right-click. <i>Recommended:</i> same colour refreshes; lifting allowed.
D4	Settled. Colour	A palette. <i>Recommended:</i> host-chosen, 2–32 colours, default sixteen; identified by number, never by name.
D5	Settled. Decay	Opacity falls per unit until refreshed or gone. <i>Recommended:</i> $1/k$ per unit u on block time; presets, default one hour by 24; faded stripes keep their place.
D6	Phases	Lobby: enter, tags, visibility, talk, pay. Active: also ink. Closed: read only.
D7	Round size	Capacity fixed at creation, 3–64, default 24; leavers keep the stripes they gave.
D8	Rewarding disclosure	The commons, the portrait and the conversation; show-to-see as a host option, off by default; a disclosure stipend later.
D9	Pacing	One ink per stripe per minute of block time; client coalescing.
D10	Trust in the relay	A trusted relay run by the Cooperative in version 1, said plainly in the client.
D11	Settled. Messages and payments	Included, exactly as F1R3Pix and F1R3Beat.
D12	Wheel order	Most recent interaction first; by name, or newest stripe on you, on request.
D13	Settled. Aggregates	Spectrum and trend, chosen by the player. <i>Recommended:</i> built only from what the viewer may see; volume counts every ink.
D14	Galleries	round (anyone, public data only) and flag (owner only), linked and verifiable.
D15	Veiling	An owner may veil stripes from others' view of a public flag.

The author of the game also settled the visibility rule of §3.3. It is not set out as a decision of its own, because D1 and D8 rest on it.

18.1 Out of scope for version 1

Trustless anonymity through anonymous credentials, once F1R3Node-Rust offers ring signatures, blind signatures or zero-knowledge membership proofs as system processes; replies from a target to an anonymous inker through the relay; disclosure stipends through sponsorships; partial disclosure, a flag public to chosen players only; tags private to some players; correlating sentiment across games; tournaments; hidden recipient lists and a report flow, as in F1R3Pix; and the F1R3Gaze version.